FACTORY CORE — STAGES 2–5 + MASTER EXECUTION GUIDE

This file is the operator's reference. Attach to every Claude Code session.

STAGE 2: VOICE PACKAGES

Claude Code Command: claude --continue --allowedTools Bash,Write,Read

Prerequisite: STAGE_1_COMPLETE.md exists at repo root

Pre-flight

```bash
cat CLAUDE.md
cat STAGE_1_COMPLETE.md
git log --oneline -5
```

Packages to build (in order)

@factory/llm

Depends on: @factory/errors, @factory/logger Priority: Build this before @factory/telephony

Full Anthropic → Grok → Groq failover chain.

```typescript
```

```typescript
export interface LLMMessage {
  role:    'user' | 'assistant' | 'system';
  content: string;
}

export interface LLMOptions {
  model?:       string;
  maxTokens?:   number;   // default 1024
  temperature?: number;   // default 0.7
  system?:      string;
}

export interface LLMResult {
  content:  string;
  provider: 'anthropic' | 'grok' | 'groq';
  tokens:   { input: number; output: number };
  latency:  number;  // ms
}

// Primary completion with automatic failover
export async function complete(
  messages: LLMMessage[],
  env: {
    ANTHROPIC_API_KEY: string;
    GROK_API_KEY:      string;
    GROQ_API_KEY:      string;
  },
  opts?: LLMOptions
): Promise<FactoryResponse<LLMResult>>

// Streaming completion (returns Anthropic stream, no failover on stream)
export async function stream(
  messages: LLMMessage[],
  env: { ANTHROPIC_API_KEY: string },
  opts?: LLMOptions
): Promise<ReadableStream>

// Pre-binds system prompt — returns a complete() with system pre-loaded
export function withSystem(system: string):
  (messages: LLMMessage[], env: LLMEnv, opts?: LLMOptions) =>
    Promise<FactoryResponse<LLMResult>>
```

Failover logic:

1. Try Anthropic (claude-sonnet-4-20250514)
2. On rate limit (429) or server error (5xx) → try Grok (grok-3-fast)
3. On Grok failure → try Groq (llama-3.3-70b-versatile)
4. All three fail → return FactoryResponse with LLM_ALL_PROVIDERS_FAILED error
5. Log each failover event via @factory/logger with provider and error code

Tests required:

- complete: returns Anthropic result on success
- complete: fails to Grok on Anthropic 429
- complete: fails to Groq on Anthropic + Grok failure
- complete: returns error response when all three fail
- withSystem: prepends system message correctly
- stream: returns ReadableStream

@factory/telephony

Depends on: @factory/errors, @factory/logger, @factory/llm

```typescript
```

```typescript
export interface VoiceSessionConfig {
  callId:       string;
  direction:    'inbound' | 'outbound';
  voiceId:      string;   // ElevenLabs voice ID
  language?:    string;   // default 'en-US'
  systemPrompt: string;
  env: {
    TELNYX_API_KEY:      string;
    DEEPGRAM_API_KEY:    string;
    ELEVENLABS_API_KEY:  string;
    ANTHROPIC_API_KEY:   string;
    GROK_API_KEY:        string;
    GROQ_API_KEY:        string;
  };
}

export interface Transcript {
  speaker:  'user' | 'agent';
  text:     string;
  ts:       number;
  duration: number;
}

export class VoiceSession extends EventTarget {
  constructor(config: VoiceSessionConfig)
  async start(): Promise<void>
  async processAudio(audio: ArrayBuffer): Promise<void>
  async end(): Promise<Transcript[]>
}

// Standalone STT — Deepgram
export async function transcribe(
  audio:     ArrayBuffer,
  apiKey:    string,
  opts?:     { language?: string; model?: string }
): Promise<string>

// Standalone TTS — ElevenLabs
export async function synthesize(
  text:     string,
  voiceId:  string,
  apiKey:   string,
  opts?:    { stability?: number; similarityBoost?: number }
```

```
): Promise<ArrayBuffer>

// Telnyx webhook handler
export async function handleTelnyxWebhook(
  request: Request,
  apiKey:  string
): Promise<Response>
```

Tests required:

- transcribe: sends correct request to Deepgram, returns string
- synthesize: sends correct request to ElevenLabs, returns ArrayBuffer
- handleTelnyxWebhook: parses inbound call event, returns 200
- VoiceSession: constructs with config, start/end lifecycle

@factory/testing

No dependencies — pure mock factories

```typescript
```

```
// All mock factories for use in app test suites

export const mockNeon: () => MockNeonDatabase
export const mockStripe: () => MockStripeClient
export const mockLLM: (responses?: string[]) => MockLLMClient
export const mockTelnyxWebhook: (event: string) => Request
export const mockVoiceSession: () => MockVoiceSession
export const mockResend: () => MockResendClient
export const mockPostHog: () => MockPostHogClient
export const mockSentry: () => MockSentryClient

export const createTestUser: (overrides?: Partial<TestUser>) => TestUser
export const createTestTenant: (overrides?: Partial<TestTenant>) => TestTenant
export const createTestSubscription: (overrides?: Partial<TestSub>) => TestSub

// Test request builder for Hono handlers
export const createTestRequest: (opts: {
  method:   string;
  path:     string;
  body?:    unknown;
  headers?: Record<string, string>;
  user?:    TokenPayload;
}) => Request
```

Stage 2 Completion

```bash
git commit -m "docs: Stage 2 complete — voice packages published"
git tag -a "stage-2-complete" -m "LLM + telephony + testing published"
git push && git push --tags
```

STAGE 3: OPERATIONS PACKAGES

Prerequisite: STAGE_2_COMPLETE.md exists

@factory/analytics

```typescript
export type EventDestination = 'posthog' | 'factory_events' | 'both';

export interface AnalyticsConfig {
  postHogKey: string;
  db:         NeonDatabase;
  appId:      string;
}

export function initAnalytics(config: AnalyticsConfig): Analytics

export interface Analytics {
  // Routes to PostHog + factory_events based on event type
  track(event: string, properties?: Record<string, unknown>, userId?: string): Promi
  // PostHog only
  identify(userId: string, traits: Record<string, unknown>): Promise<void>;
  // factory_events only — revenue and compliance events
  businessEvent(event: string, properties: Record<string, unknown>, userId?: string)
  // Page view for PostHog funnels
  page(name: string, properties?: Record<string, unknown>): Promise<void>;
}
```

@factory/deploy (scripts package — no TypeScript source)

Contents:

- `scripts/deploy.sh` — wrangler deploy with env validation + Sentry release tagging
- `scripts/setup-secrets.sh` — interactive secret configuration for new apps
- `scripts/validate-env.sh` — checks all required env vars are set
- `templates/ci.yml` — GitHub Actions CI template for apps
- `templates/deploy.yml` — GitHub Actions deploy template for apps
- `templates/wrangler.jsonc` — wrangler config template
- `templates/.env.factory.example` — all required env var names with comments
- `README.md` — usage for each script

@factory/monitoring (additions to Phase 1 base)

Add to existing package:

- Hono middleware that wraps entire app in Sentry performance transaction
- `withPerformance()` wrapper for any async function
- Automatic Worker name detection from env binding

STAGE 4: MARKETING PACKAGES

Prerequisite: STAGE_3_COMPLETE.md exists

@factory/email

```typescript
```

```typescript
export interface EmailConfig {
  resendApiKey: string;
  fromAddress:  string;   // e.g. 'noreply@thefactory.dev'
  fromName:     string;
}

export function createEmailClient(config: EmailConfig): EmailClient

export interface EmailClient {
  sendTransactional(opts: {
    to:       string;
    subject:  string;
    html:     string;
    text?:    string;
    replyTo?: string;
  }): Promise<{ id: string }>;

  enrollDrip(opts: {
    userId:   string;
    email:    string;
    sequence: string;
    data?:    Record<string, unknown>;
  }): Promise<void>;

  unsubscribe(userId: string, email: string): Promise<void>;
}
```

@factory/copy

Brand voice registry + Claude API generation with app-scoped voice profiles.

```typescript
```

```typescript
export interface VoiceProfile {
  tone:        string[];   // e.g. ['dignified', 'warm', 'encoded']
  vocabulary:  string[];   // preferred words
  avoid:       string[];   // words/phrases to never use
  register:    'formal' | 'professional' | 'conversational' | 'casual';
  example:     string;     // example sentence in this voice
}

export const voiceProfiles: Record<string, VoiceProfile> = {
  'cypher_healing': { ... },   // Classic Man — dignified, encoded, warm
  'prime_self':     { ... },   // Precise, archetypal, grounded
  'ijustus':        { ... },   // Professional, direct, enterprise
  'the_calling':    { ... },   // Energetic, competitive, fun
  'default':        { ... },   // Neutral Factory voice
};

export function registerVoice(appId: string, profile: VoiceProfile): void
export function getVoiceProfile(appId: string): VoiceProfile
export async function generateCopy(opts: {
  prompt:  string;
  appId:   string;
  env:     { ANTHROPIC_API_KEY: string; GROK_API_KEY: string; GROQ_API_KEY: string }
  maxLen?: number;
}): Promise<string>
```

@factory/seo

```typescript
```

```typescript
export function generateMetaTags(opts: {
  title:       string;
  description: string;
  url:         string;
  image?:      string;
  type?:       'website' | 'article' | 'product';
}): string   // returns HTML meta tag string

export function generateSitemap(pages: Array<{
  url:       string;
  priority?: number;
  changefreq?: 'daily' | 'weekly' | 'monthly';
}>): string   // returns XML sitemap string

export function generateJsonLd(type: 'Organization' | 'SoftwareApplication' | 'Servi
  data: Record<string, unknown>): string   // returns JSON-LD script tag
```

@factory/content + @factory/social

- @factory/content: Neon-backed content calendar with status state machine (draft → review → approved → queued → published → archived)
- @factory/social: Wrapper over social platform APIs where available (X/Twitter API v2, Pinterest API). TikTok and Instagram: document that these require the VPS-hosted Appium layer — provide webhook interface only.

STAGE 5: REVENUE OPS PACKAGES

Prerequisite: STAGE_4_COMPLETE.md exists

@factory/crm

```typescript
```

```typescript
export interface Lead {
  id:         string;
  userId:     string;
  appId:      string;
  source:     string;   // 'organic' | 'tiktok' | 'referral' | etc.
  status:     'lead' | 'trial' | 'active' | 'churned';
  mrr:        number;   // monthly recurring revenue in cents
  createdAt:  Date;
  convertedAt?: Date;
}

export function trackLead(db: NeonDatabase, opts: {
  userId: string; appId: string; source: string;
}): Promise<Lead>

export function trackConversion(db: NeonDatabase, opts: {
  userId: string; plan: string; mrr: number;
}): Promise<void>

export function getCustomerView(db: NeonDatabase, userId: string): Promise<{
  lead:          Lead;
  subscriptions: SubscriptionStatus[];
  events:        FactoryEvent[];
  churnRisk:     'low' | 'medium' | 'high';
}>
```

@factory/compliance

typescript

```typescript
// TCPA check — returns true if safe to contact
export async function checkTCPA(opts: {
  phone: string;
  db:    NeonDatabase;
}): Promise<{ safe: boolean; reason?: string }>

// Immutable consent record
export async function logConsent(db: NeonDatabase, opts: {
  userId:      string;
  consentType: 'TCPA' | 'FDCPA' | 'GDPR' | 'CCPA';
  ipAddress:   string;
  userAgent?:  string;
}): Promise<void>

// FDCPA contact validation
export async function checkFDCPA(db: NeonDatabase, opts: {
  contactId:  string;
  callType:   'initial' | 'follow_up';
}): Promise<{ allowed: boolean; nextAllowedAt?: Date; reason?: string }>
```

@factory/admin

Hono router that every app mounts at /admin.

```typescript
export function createAdminRouter(opts: {
  db:        NeonDatabase;
  analytics: Analytics;
  appId:     string;
}): Hono

// Mounts at /admin in the host app:
// GET  /admin              → dashboard summary
// GET  /admin/users        → paginated user list
// GET  /admin/users/:id    → user detail with subscription + events
// POST /admin/users/:id/suspend
// GET  /admin/events       → recent factory_events for this app
// GET  /admin/health       → app health check with DB ping
```

Full Command Reference

```bash
# Start fresh session (Stage 0)
claude --allowedTools Bash,Write,Read

# Continue after interruption (same session)
claude --continue --allowedTools Bash,Write,Read

# Resume specific session by ID
claude --resume <session-id> --allowedTools Bash,Write,Read

# Monitor output programmatically
claude --continue --allowedTools Bash,Write,Read \
  --output-format stream-json | tee build-log.json
```

Stage Trigger Checklist

Before starting each stage, verify:

```bash
# Is the prerequisite complete file present?
ls STAGE_<N-1>_COMPLETE.md

# Are all previous packages published?
gh api /orgs/thefactory/packages?package_type=npm | jq '.[].name'

# Is CI green on main?
gh run list --branch main --limit 5 --status success

# Is CLAUDE.md up to date?
cat CLAUDE.md | head -20
```

When Things Go Wrong

TypeScript errors after npm install:

```bash
```

```bash
# Clear caches and reinstall
rm -rf node_modules package-lock.json
npm install
npm run typecheck
```

Coverage below threshold:

```bash
# Find uncovered lines
npm run test -- --coverage --reporter=verbose
# Add tests for uncovered branches — never lower the threshold
```

Publish fails (auth error):

```bash
# Verify GitHub Packages auth
npm whoami --registry https://npm.pkg.github.com
# Re-authenticate if needed
npm login --registry https://npm.pkg.github.com \
  --auth-type legacy
```

Worker constraint violation caught by ESLint:

```bash
# Read the specific rule violation
npm run lint -- --format verbose
# Fix the source — never add eslint-disable comments
# Check CLAUDE.md Hard Constraints section for correct alternative
```

Context window filling mid-package:

```bash
```

```
# Commit work in progress
git add -A
git commit -m "wip(<package>): <describe what's done>"
git push

# Resume with focused context
claude --resume <session-id> --allowedTools Bash,Write,Read
# First message: "Read CLAUDE.md, then git log --oneline -3.
#   Continue building @factory/<package> from where we left off."
```

Quality Gate Automation Script

Save this as `scripts/quality-gate.sh` in the repo:

```bash
#!/bin/bash
set -e
PACKAGE=$1
if [ -z "$PACKAGE" ]; then echo "Usage: ./scripts/quality-gate.sh <package-name>"; e

echo "=== Quality Gate: @factory/$PACKAGE ==="
cd packages/$PACKAGE

echo "→ Lint..."
npm run lint

echo "→ TypeCheck..."
npm run typecheck

echo "→ Test + Coverage..."
npm run test

echo "→ Build..."
npm run build

echo "✅ All gates passed for @factory/$PACKAGE"
```

```bash
chmod +x scripts/quality-gate.sh
# Usage: ./scripts/quality-gate.sh errors
```

Publishing Tag Convention

```bash
# Tag format: <package>/v<version>
git tag errors/v0.1.0
git tag logger/v0.1.0

# Publish is triggered automatically by GitHub Actions on tag push
git push --tags
```

Estimated Timeline (actual working time, not wall clock)

```
Stage 0 — Bootstrap:         ~2 hours
Stage 1 — Foundation (6):    ~8 hours
Stage 2 — Voice (3):         ~6 hours
Stage 3 — Operations (3):    ~4 hours
Stage 4 — Marketing (5):     ~6 hours
Stage 5 — Revenue Ops (3):   ~5 hours
──────────────────────────────────────
Total:                        ~31 hours working time
Across 2-3 week sprint with review gates between stages
```

App Migration Order (after Factory Core is complete)

Once all packages are published, migrate existing apps in this order:

1. **FocusBro** — simplest, no voice, no billing complexity. Proof of concept.
2. **Prime Self Engine** — high value, establishes the voice + billing pattern
3. **iJustus** — voice + billing + multi-tenant. Full pattern validation.
4. **CypherOfHealing** — five revenue streams. Tests everything.
5. **The Calling** — voice game. Tests telephony package at scale.
6. **Wordis Bond** — compliance package required. Build after Stage 5.

Migration per app:

```bash
```

```
npm install @factory/errors @factory/logger @factory/auth \
            @factory/neon @factory/stripe @factory/monitoring \
            @factory/analytics
# Replace existing implementations one module at a time
# Run tests after each replacement
# Never big-bang migrate — incremental only
```

FACTORY CORE IS COMPLETE WHEN

```
✅ 19 packages published to GitHub Packages at v0.1.0+
✅ All packages: 90%+ test coverage
✅ All packages: zero TypeScript strict errors
✅ All packages: zero ESLint warnings
✅ CI green on main for all packages
✅ Mintlify docs deployed for all packages
✅ Renovate bot active and monitoring all app repos
✅ STAGE_5_COMPLETE.md committed to main
✅ FocusBro migrated as proof-of-concept app
```

At that point: every new Factory app scaffolds in under 2 hours.
Every app sale closes cleanly with one repo transfer.
Every hour you used to spend rebuilding plumbing is returned to product.